COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:		**SECTION:**	CPE21S1
DATE PERFORMED:	08/27/2026	**DATE SUBMITTED:**	08/27/2026
NAME:	Jarel Dave R. Egar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILO A

Screenshot | main.cpp

Projects_Cpp > HOA6p1 > main.cpp > Student

```cpp
#include <iostream>
#include <string>
#include "searchingAlgo.h"

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    // Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor was called" << std::endl;
    }

    // Destructor
    ~Student() {
        std::cout << "Destructor was called" << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Display attributes
    void printDetails() {
        std::cout << this->studentName << " "
                  << this->studentAge << std::endl;
    }
};
```

```cpp
int main() {

    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {
        "Dane", "Angela", "Andrew", "Ralph", "Jarel"
    };
    int ageList[j] = {
        15, 16, 18, 19, 16
    };

    // Create Student objects
    for (int i = 0; i < j; i++) {
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;

        delete ptr;
    }

    // Display students
    std::cout << "\nStudent List:\n";

    for (int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }


    // LINEAR SEARCH - ARRAY

    int data[] = {10, 20, 30, 40, 50};
    int arrSize = 5;

    std::cout << "\nLinear Search Array:\n";

    linearSearchArr(data, arrSize, 30);
    linearSearchArr(data, arrSize, 100);

```

```cpp
    // LINEAR SEARCH - LINKED LIST


    Node<int>* head = new Node<int>(10);
    head->next = new Node<int>(20);
    head->next->next = new Node<int>(30);
    head->next->next->next = new Node<int>(40);
    head->next->next->next->next = new Node<int>(50);

    std::cout << "\nLinear Search Linked List:\n";

    linearSearchLL(head, 30);
    linearSearchLL(head, 100);


    // BINARY SEARCH - LINKED LIST


    std::cout << "\nBinary Search Linked List:\n";

    binarySearchLL(head, 40);
    binarySearchLL(head, 100);


    // DELETE LINKED LIST

    Node<int>* current = head;

    while (current != nullptr) {
        Node<int>* next = current->next;
        delete current;
        current = next;
    }

    return 0;
}
```

Screenshot | nodes.h

main.cpp | node.h | searchingAlgo.h

Projects_Cpp > HOA6p1 > node.h > ...

```cpp
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node {
    public:
        T data;
        Node* next;

};

template <typename T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>();
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

#endif
```

Screenshot | SearchingAlgo.h

main.cpp | node.h | searchingAlgo.h

Projects_Cpp > HOA6p1 > searchingAlgo.h > ...

```cpp
#include <iostream>
#include "node.h"

#ifndef SEARCHING_ALGO_H
#define SEARCHING_ALGO_H

template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind){
    int i = 0;

    while(i < arrSize){
        if(dataFind == data[i]){
            std::cout << "the search is success\n";
            return;
        }
        i++;
    }
    std::cout << "the element is not found \n";
    return;
}

template <typename T>
void linearSearchLL(Node<T>* head, T dataFind){
    while(head != nullptr){
        if(dataFind == head->data){
            std::cout << "the search was a success\n";
            return;
        }
        head = head->next;
    }
    std::cout << "the search was unsuccessful\n";
}
```

```cpp
// FIXED: Added '*' to 'last' parameter so it matches Node<T>*
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){
    if (start == nullptr) return nullptr;

    Node<T>* slow = start;
    Node<T>* fast = start;

    // Safety check added: fast->next must not be null before skipping two steps
    while(fast != last && fast->next != last){
        slow = slow->next;
        fast = fast->next;
        if (fast != last) {
            fast = fast->next;
        }
    }

    return slow;
}
```

```cpp
template <typename T>
Node<T>* binarySearchLL(Node<T>* head, T dataFind){
    Node<T>* start = head;
    Node<T>* last = nullptr;

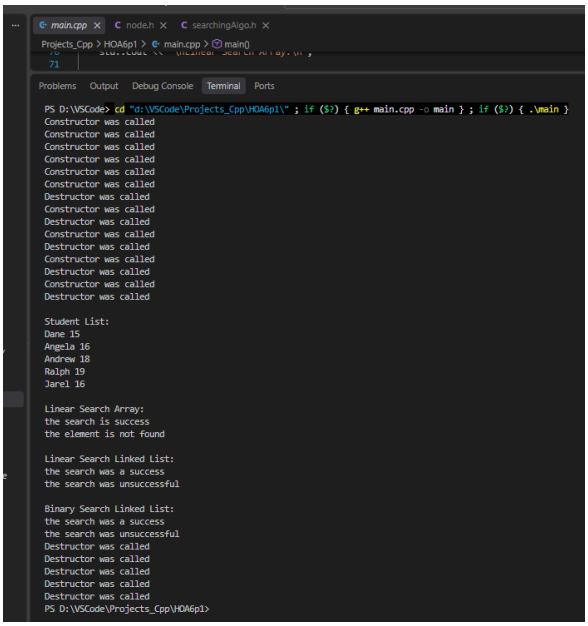
    // Standard condition: continue searching as long as search space is valid
    while (start != last) {
        Node<T>* mid = getMiddle(start, last);

        if(mid == nullptr){
            break;
        }

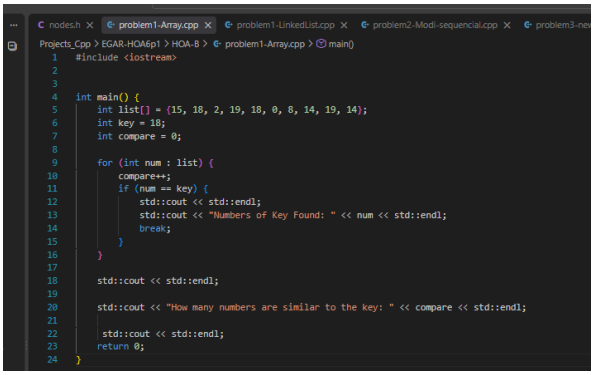
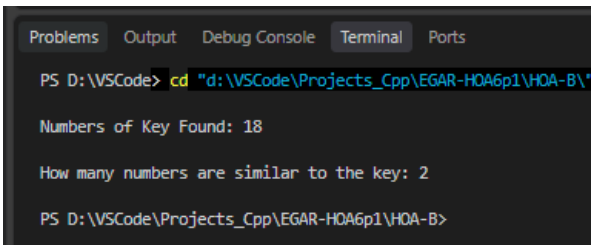
        if(mid->data == dataFind){
            std::cout << "the search was a success\n";
            return mid;
        }
        // Key is larger -> search right half
        else if(mid->data < dataFind){
            start = mid->next;
        }
        // Key is smaller -> search left half
        else{
            last = mid;
        }
    }

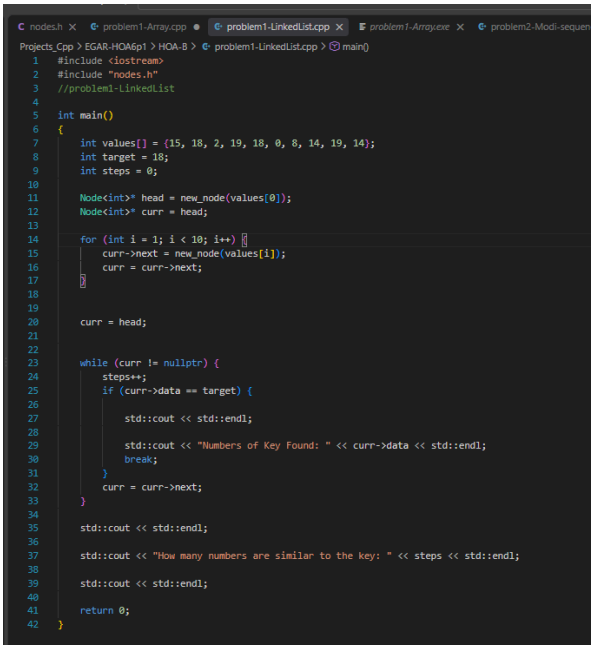
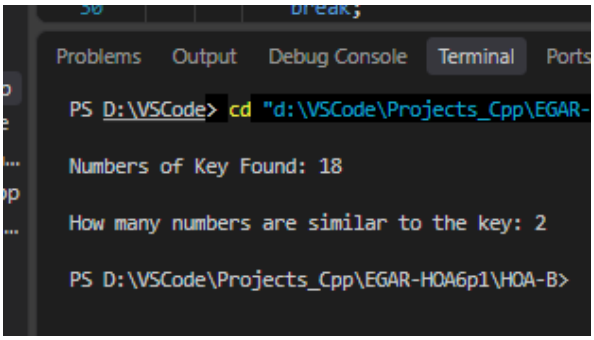
    std::cout << "the search was unsuccessful\n";
    return nullptr;
}

#endif
```

Output	

ILO B / SUPPLEMENTARY

Screenshot | problem1-Array	
Output	

Screenshot | problem1-LinkedList	
Output	

COMPUTER ENGINEERING

QUEZON CITY

Screenshot | problem2-Modi-sequencial

Projects_Cpp > EGAR-HOA6p1 > HOA-B > problem2-Modi-sequencial.cpp > ...

```cpp
#include <iostream>
#include "nodes.h"
//problem2-Modi-sequencial

int countArray(int arr[], int size, int target)
{
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            count++;
        }
    }
    return count;
}

int countLinkedList(Node<int>* head, int target)
{
    int count = 0;
    Node<int>* current = head;

    while (current != nullptr) {
        if (current -> data == target) {
            count++;
        }
        current = current -> next;
    }
    return count;
}

int main()
{
    int values[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int k = 18;


    Node<int>* head = new_node(values[0]);
    Node<int>* current = head;
    for (int i = 1; i < 10; i++) {
        current -> next = new_node(values[i]);
        current = current -> next;
    }


    int arrayResult = countArray(values, 10, k);
    int listResult = countLinkedList(head, k);

    std::cout << std::endl;

    std::cout << "Array count for key '" << k << "': " << arrayResult << std::endl;

    std::cout << std::endl;

    std::cout << "Linked List count for key '" << k << "': " << listResult << std::endl;

    std::cout << std::endl;

    return 0;
}
```

Output

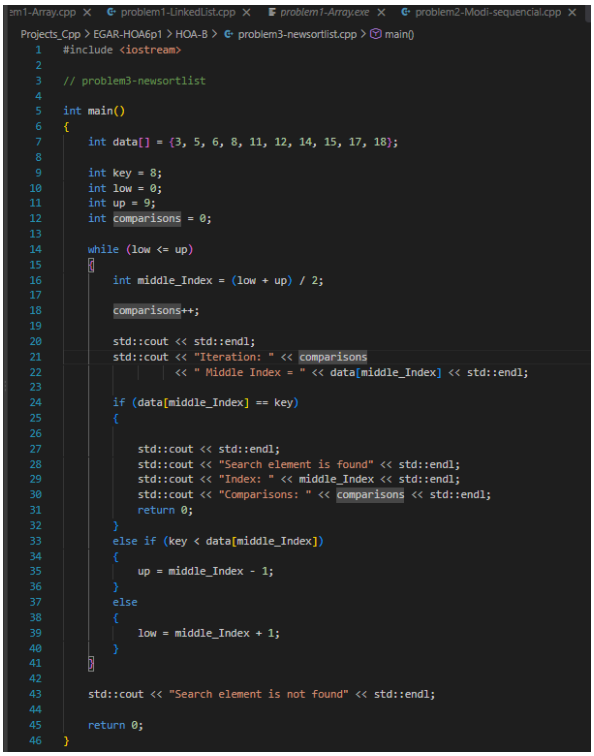
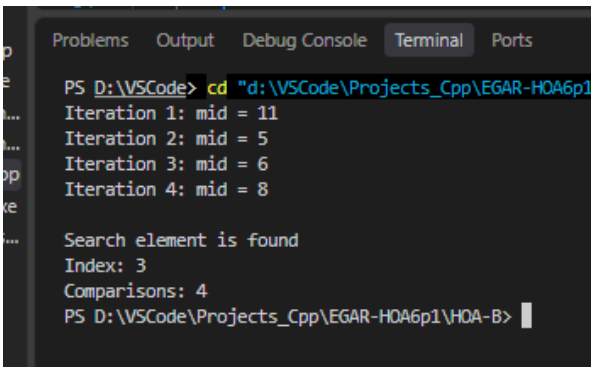
```
    return 0;
}
Problems  Output  Debug Console  Terminal  Ports

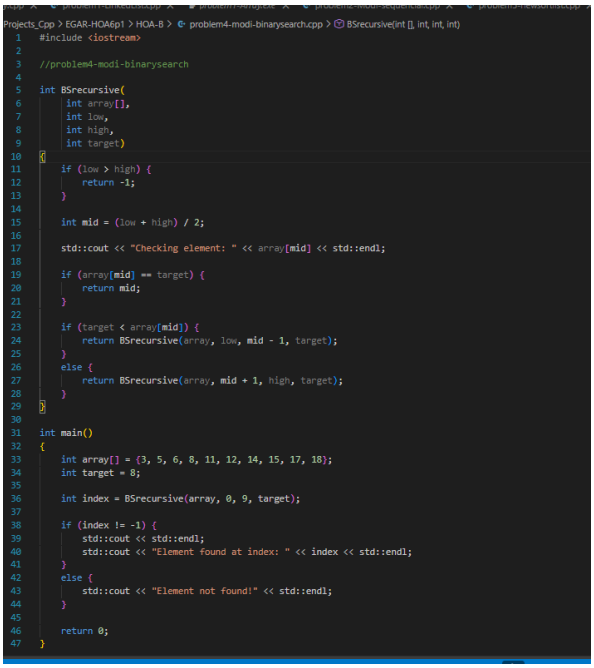
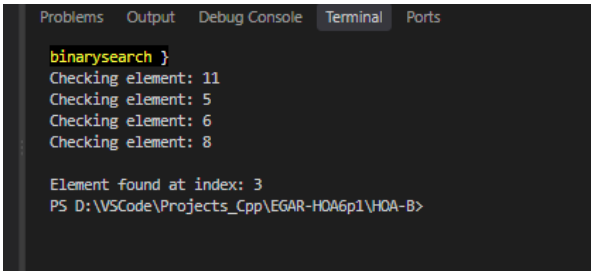
PS D:\VSCode> cd "d:\VSCode\Projects_Cpp\EGAR-HOA6p1\
encial }

Array count for key '18': 2

Linked List count for key '18': 2

PS D:\VSCode\Projects_Cpp\EGAR-HOA6p1\HOA-B>
```

Screenshot | problem3-newsortlist	
Output	

Screenshot | problem4-modi-binarysearch	
Output	

CONCLUSION & CODE ANALYSIS: During the activity I learned how to use searching techniques by using the binary and sequential search. During problem 1 I noticed that since it is required to use an array, it checks each index one by one then after it finishes checking it will display the key 18 which is the duplicate in the index. In the 2nd problem it has the same function as problem 1 but the difference is that problem 2 does not stop when it finds the first duplicated number but it continues to search the array until the very end and returns what number has a duplicate. In problem 3 I learned how to search a list by using binary search, first the search will start in the middle until it finds the desired key which is 8. Lastly problem 4, I modified problem 3 to match the question then I noticed that the recursive stack stores each recursive call, changing the high and low index number until it reaches key 8.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini to help in explaining how the error happened and what I can do to fix it but I asked gemini to not give me the code, just the idea of how to fix it.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.